

Haiku reads, Opus plans, Sonnet builds, Opus
reviews: my Claude Code subagent setup

Kai Du

2026-10-03

For a while I ran Claude Code the lazy way: one strong model for everything. It worked, and it was expensive in the way that only shows up when the usage window runs out at 3 pm. Most of what a coding agent does is not hard. It greps, opens files and skims them. The hard parts are deciding what to change and checking afterwards that the change is right. So I split the work by difficulty and gave each stage the model that suits it.

This post describes that setup, what I measured when I tested it, what I changed after the first draft failed in small ways, and how I stop it from going stale. Everything was tested on Claude Code 2.1.285 on 3 October 2026. Claude Code moves quickly, so check the field names against the current documentation before you copy anything.

The idea in one table

Stage	Who	How it is wired
Read	Haiku , as a custom Explore agent	A file in <code>~/.claude/agents/</code> that overrides the built-in Explore. Read-only
Plan	Opus , main session in plan mode	The opusplan model alias
Build	Sonnet , main session after you approve the plan	The same opusplan alias switches models when plan mode ends
Review	Opus , as a custom reviewer agent	A second file in <code>~/.claude/agents/</code> . Read-only
Guard	A hook script	Allows only read-only shell commands for both

What the two agents look like

The reader is the cheap one. This is the top of its file:

```
name: Explore
description: Fast read-only codebase reader. Use before pla
tools: Read, Grep, Glob, Bash
model: haiku
omitClaudeMd: true
maxTurns: 30
```

Using the same name as the built-in agent overrides it, and the match is case-sensitive. `omitClaudeMd: true` stops it loading my large instruction files, which a cheap reader does not need. The body of the file is a mandatory output template, and I will come back to why.

The reviewer is the expensive one:

How I use it day to day

1. Start claude. The default model is already opusplan. Enter plan mode with Shift+Tab, /plan, or --permission-mode plan.
2. Describe the task. Opus hands the reading to Explore and writes a plan. I read the plan and approve it, and Sonnet implements.
3. When the work is done, I ask for the reviewer: “use the reviewer subagent”. **I paste in the plan and anything I have already decided.** A subagent does not see the conversation, so an unbriefed reviewer will cheerfully re-argue questions that were settled an hour ago.
4. I fix what it reports and commit.

Some escape hatches: if Sonnet fails the same step twice, /model opus for that step and then /model opusplan to go back. For a side task that needs the conversation so far, use /subtask <instruction>; the fork runs on the main session's model, which is Sonnet during execution. On a very large task, re-enter plan mode after each milestone so Opus plans from the current state

The part where Haiku is the weak link

The design has an obvious risk. Opus plans from what Haiku reports, so a confident but incomplete brief becomes a confident but incomplete plan. I therefore made the explorer say what it did not cover. Every brief must have FACT and INFERENCE labels with a path:line citation, and must end with two lists: NOT FOUND (things it looked for and could not find) and NOT READ (relevant-looking files it did not open).

My first version of that prompt was a polite request, and Haiku followed it in 1 of 3 test briefs. Replacing the request with a fill-in template made it 3 of 3. On three blind questions with an answer key, there were no fabrications, every cited line was right, and the one trap question was answered correctly. One brief used absolute paths, and I still treat NOT READ: none as a claim to spot-check. Three questions is a thin sample; the real test is three real tasks, and I have not done those yet.

What the reviewer is told to check

I wanted the reviewer to behave like a skeptical colleague, so its prompt is an ordered checklist: did the change match the plan; is there a silent numerical error (a sign, an off-by-one index, a reseeded random number generator, a changed constant); can results still be traced back to the code that produced them; did anything break at the call sites; is anything unverified; and only then, hygiene. Every finding must be marked MEASURED or INFERRED, and “nothing wrong” must come with a list of what was checked. The verdict is one of APPROVE, APPROVE WITH NOTES or CHANGES REQUESTED.

What changed after the first draft

I wrote the setup from an initial draft and then checked it against the documentation and an end-to-end test. The changes I would flag for anyone doing the same:

- ▶ **Read-only by prompt is not read-only.** My draft told the reviewer's shell to be read-only in its instructions. A prompt is advice, not a control. I added a `PreToolUse` hook on Bash that allows a short list of read-only commands and refuses everything else with exit code 2. In a test, the reviewer was refused on `echo hi > file` and on `git commit`, while `git status` and `git log` ran without a permission prompt. A 36-case unit battery of the guard passed (10 allow, 26 deny). It is a guardrail against accidents and prompt drift, not a sandbox: it does not stop the agent reading sensitive files, because read access is the point.
- ▶ **The explorer needed a shell after all.** I had given it only `Read`, `Grep`, `Glob`, which lost `git log` and `git blame`. I added Bash back, behind the same guard.
- ▶ **No tests from the reviewer** An allowlist cannot know each

What I have not measured

I would rather list these than imply the setup is proven:

- ▶ The live `opusplan` handover when you approve a plan interactively. Headless runs cannot approve a plan, though I did confirm that plan mode ran Opus 5.5 and execution ran Sonnet 5.5.
- ▶ Haiku's brief quality over real work, as above.
- ▶ Whether per-model effort settings are honoured per phase under `opusplan`. The documentation says effort carries across phases.

Keeping it from going stale

A setup like this decays quietly: an unknown frontmatter field is silently ignored, so a renamed field fails without an error. I run a review every 30 days, and on any minor release or new model. A small install script reports whether the installed copies match my canonical ones, whether the review is overdue, and whether the Claude Code version moved. The review itself is short:

1. Re-read the two documentation pages below, and check that the field names, the model aliases and the descriptions of `opusplan` and the `Explore` override are unchanged.
2. Re-run the self-test: have the reviewer try `git status`, an `echo` redirect and a `git commit`, and read the model IDs out of the `stream-json` output. Expect the first allowed, the others blocked, and both Haiku and Opus present. Also ask a fresh session to list its subagent types and expect exactly one `Explore` and one `reviewer`; a duplicate name in one directory loads non-deterministically.
3. Check quality, not just plumbing: verify the `path:line` citations in three recent explorer briefs, and ask of the last

What I would tell you to take away

Route by difficulty, not by habit. Make the cheap agent report its blind spots, because the expensive one will trust it. Treat a prompt as advice and use a hook where you need a guarantee. Brief the reviewer, because it starts with no memory of what you decided. And date your setup, so that you know when to distrust it.

Sources

- ▶ Claude Code documentation on subagents (frontmatter fields, scope priority, overriding the built-in agents, hooks, /subtask, nesting) and on model configuration (opusplan, aliases, effort, modelSettings). These are the authority.
- ▶ Secondary write-ups I found useful for ideas: MindStudio on saving tokens with Opus plan mode, Tembo, Glukhov and Daily Dose of DS.